

Computational Validation of Adaptive Interface Control in TMI Quantum Systems

Расширенный отчёт по вычислительным тестам 1-22

Salkutsan Aleksey

6 мая 2026

Аннотация

Этот отчёт объединяет вычислительную линию Tests 1-22 для прикладного квантового модуля Теории многообразных интерфейсов (ТМИ). Первоначальная линия Tests 1-11 проверяла рост T_2^{eff} и $P_{success}$, затем воспроизводимость и независимую реализацию были добавлены в Tests 12-13. Tests 14-19 исследуют типы шума, восстановление amplitude damping, классификацию шума, online drift, задержанную обратную связь и оптимизацию latency-aware политики. Tests 20-22 переводят управление из single-metric режима к cost boundary и multi-objective boundary manifold.

Главная проверяемая цепочка остаётся узкой:

$$I_Q^{coh-auto} \Rightarrow T_2^{eff} \uparrow \Rightarrow P_{success} \uparrow.$$

Новая расширенная цепочка:

single metric $\rightarrow$ cost boundary $\rightarrow$ multi-objective boundary manifold.

Это не лабораторное доказательство существования Ξ_I^{auto} , а воспроизводимая вычислительная линия проверки инженерного механизма автоинтерфейсного управления.

Содержание

1 Статус отчёта	3
2 Сводка Tests 1-22	3
3 Tests 1-11: базовая линия, отказ и восстановление	4
4 Tests 12-13: воспроизводимость и независимая реализация	4
4.1 Test 12: reproducibility harness	4
4.2 Test 13: independent simulator implementation	5

5 Tests 14-17: noise-specific и online adaptive interface	5
6 Tests 18-19: задержка обратной связи и latency-aware управление	8
7 Tests 20-22: от single metric к boundary manifold	9
7.1 Test 20: cost-overhead tradeoff	9
7.2 Test 21: cost-interface boundary	10
7.3 Test 22: multi-objective boundary manifold	11
8 Интерпретация для Теории многообразных интерфейсов	13
9 Главный вывод	13
10 Ограничения	14
11 Следующие шаги	14
A Состав архива	14

1. Статус отчёта

Этот документ фиксирует модуль:

TMI Quantum Computational Validation 0.2

Он должен рассматриваться как Supplement к физическому слою TMI-Physics 2.2, а не как самостоятельное доказательство всей теории.

Главная позитивная формула:

$$I_Q^{coh-auto} \Rightarrow T_2^{eff} \uparrow \Rightarrow P_{success} \uparrow$$

Главная отрицательная формула:

AutoInterface $\nRightarrow$ SafeInterface

Главная новая формула Tests 20-22:

AutoInterface + BoundaryManifoldDetection $\Rightarrow$ ManifoldAdaptiveControl

2. Сводка Tests 1-22

Test	Название	Критерий / смысл	Результат
1	Ramsey toy model	$T_2^{TMI} > T_2^{std}$	$\Delta T_2 = 58.1 \mu s$
2	Noisy Ramsey learning	$T_2^{TMI} > T_2^{std}$	$\Delta T_2 = 39.3 \mu s$
3	Small quantum circuit	$P_{TMI} > P_{std}$	$\Delta P = 0.115$
4	Stabilized interface	$P_{stab} > P_{std}$	$\Delta P = 0.026$
5	Surrogate Monte Carlo	$E[P_{TMI}] > E[P_{std}]$	$\Delta = 0.0510, Pr = 0.864$
6	Ablation study	Structured adaptive control	full TMI $\Delta = 0.0680$
7	Density-matrix MC	Full matrix validation	$\Delta = 0.0930, Pr = 0.933$
8	Cross-model validation	Multiple circuit families	$\Delta = 0.0660, Pr = 0.846$
9	Failure-boundary map	Weak/failure regions	failure=0, weak=6
10	Adversarial failure	Force real failure	failure cells=58
11	Safety recovery	Mitigation after failure	recovered=31/58
12	Reproducibility harness	One-command validation	passed=11/11
13	Independent simulator	Qiskit/Cirq-style check	$\Delta = 0.1121, Pr = 1.000$

Test	Название	Критерий / смысл	Результат
14	Noise-model stress	Noise class robustness	passed=3/4, $\Delta = 0.0068$
15	Amplitude damping recovery	T_1 -aware layer	$\Delta_{relax} = 0.1532$
16	Noise classifier	Classify then control	acc=0.952, $\Delta = 0.0554$
17	Online noise drift	Dynamic reclassification	$\Delta = 0.0255$, acc = 0.816
18	Delayed feedback	Latency hurts online control	online=0.883, delayed2=0.861
19	Latency policy optimization	Predictive latency layer	opt=0.875 > d2=0.861
20	Cost-overhead tradeoff	Utility and efficiency	$U_{hp} = 0.0705$
21	Cost-interface boundary	B_{cost}	$\lambda^* = 0.0636$
22	Multi-objective control	$\mathcal{M}_{boundary}$	std=0.608, low=0.373 ca-

3. Tests 1-11: базовая линия, отказ и восстановление

Tests 1-4 показывают переход от Ramsey-когерентности к вероятности успеха малой схемы:

$$T_2^{eff} \uparrow \Rightarrow P_{success} \uparrow.$$

Tests 5-8 показывают статистическую устойчивость: surrogate Monte Carlo, full density-matrix Monte Carlo и cross-model validation. Tests 9-11 добавляют научную честность: карта слабых зон, adversarial failure и safety-aware recovery.

4. Tests 12-13: воспроизводимость и независимая реализация

4.1. Test 12: reproducibility harness

Test 12 фиксирует репозиторный принцип:

$$\text{one command} \Rightarrow \text{summary} \Rightarrow \text{pass/fail matrix} \Rightarrow \text{manifest}.$$

Результат:

11/11 tests passed.

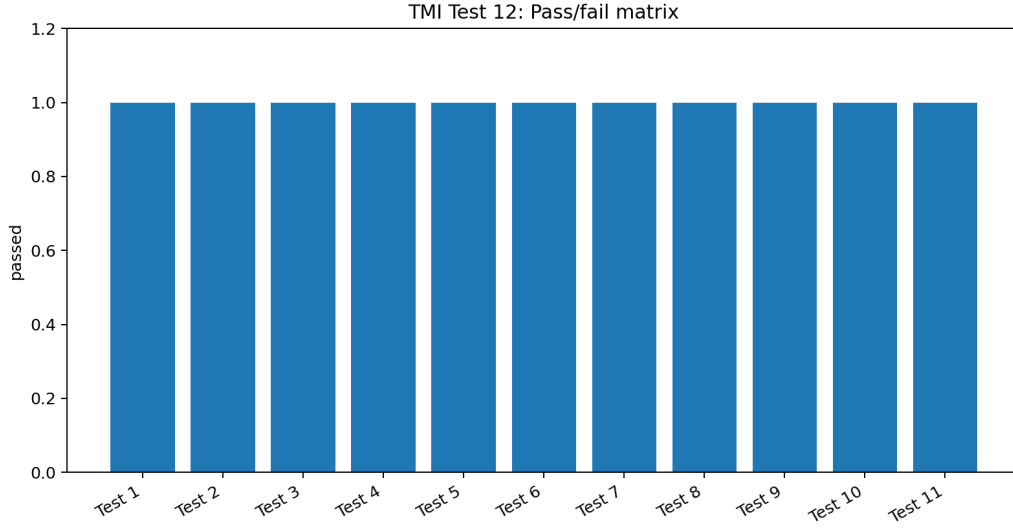


Рис. 1: Test 12: pass/fail матрица воспроизводимости.

4.2. Test 13: independent simulator implementation

Test 13 проверяет ключевой результат независимой Qiskit/Cirq-style реализацией. Результат:

$$\Delta P_{TMI-std} = 0.1121, \quad Pr(P_{TMI} > P_{std}) = 1.000.$$

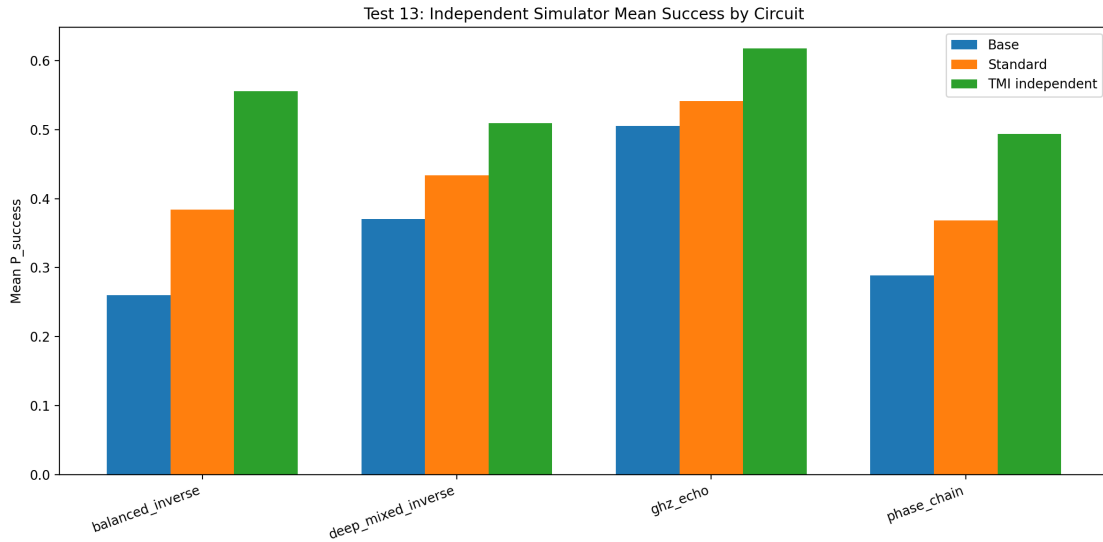


Рис. 2: Test 13: независимый симулятор подтверждает выигрыш TMI.

5. Tests 14-17: noise-specific и online adaptive interface

Test 14 показывает, что coherence-only TMI не универсален: amplitude damping требует отдельного relaxation-aware слоя. Test 15 добавляет такой

слой. Tests 16-17 развивают это в noise-classifier и online drift adaptation:

$$\text{NoiseFeatures} \Rightarrow \text{NoiseClass} \Rightarrow \text{ControlLayer} \Rightarrow P_{\text{success}} \uparrow.$$

Динамическая версия:

$$\text{NoiseClass}(t) \Rightarrow \text{ControlLayer}(t) \Rightarrow \text{OnlineAdaptiveControl}.$$

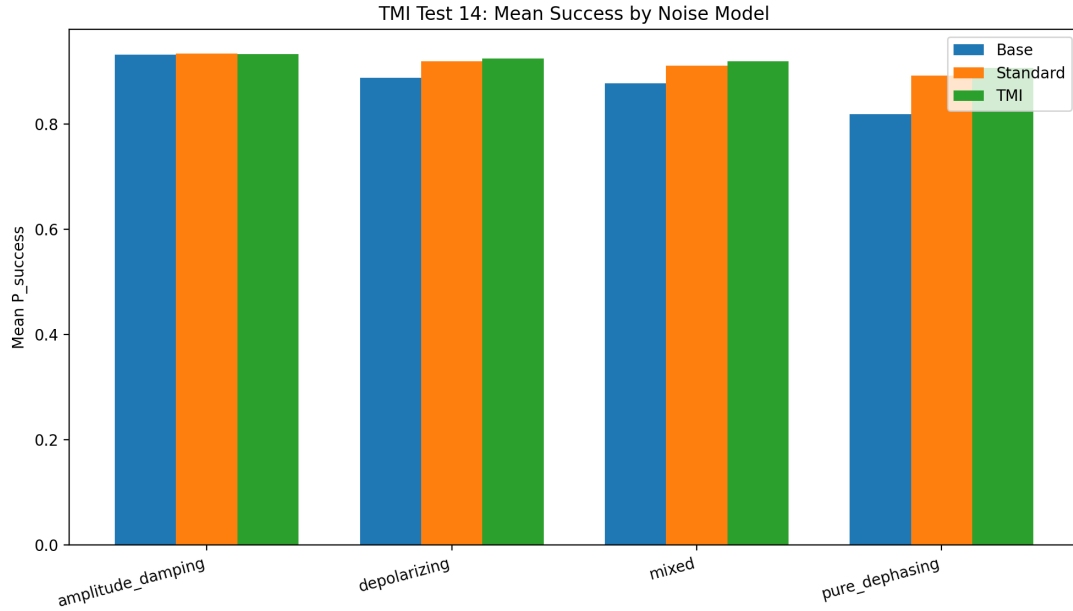


Рис. 3: Test 14: stress test по типам шума. Amplitude damping выявляет ограничение coherence-only слоя.

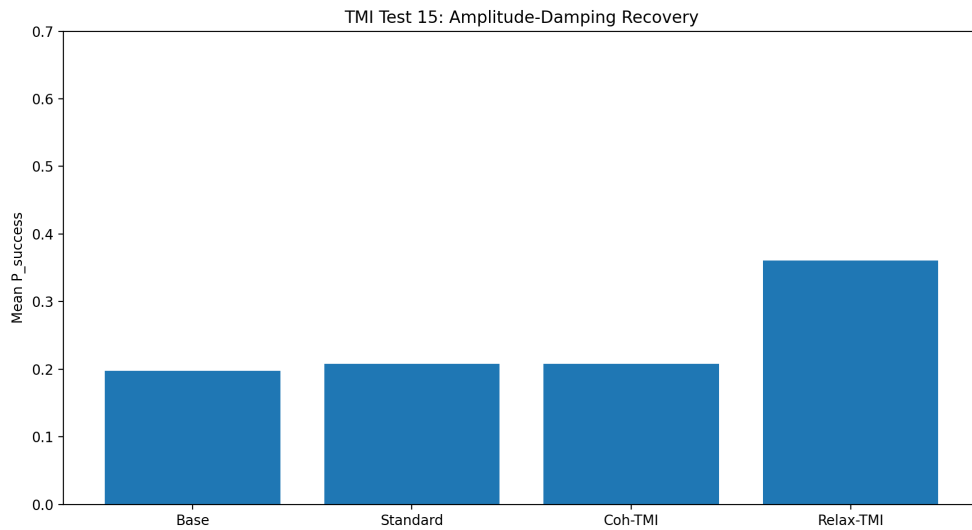


Рис. 4: Test 15: relaxation-aware слой восстанавливает выигрыш при amplitude damping.

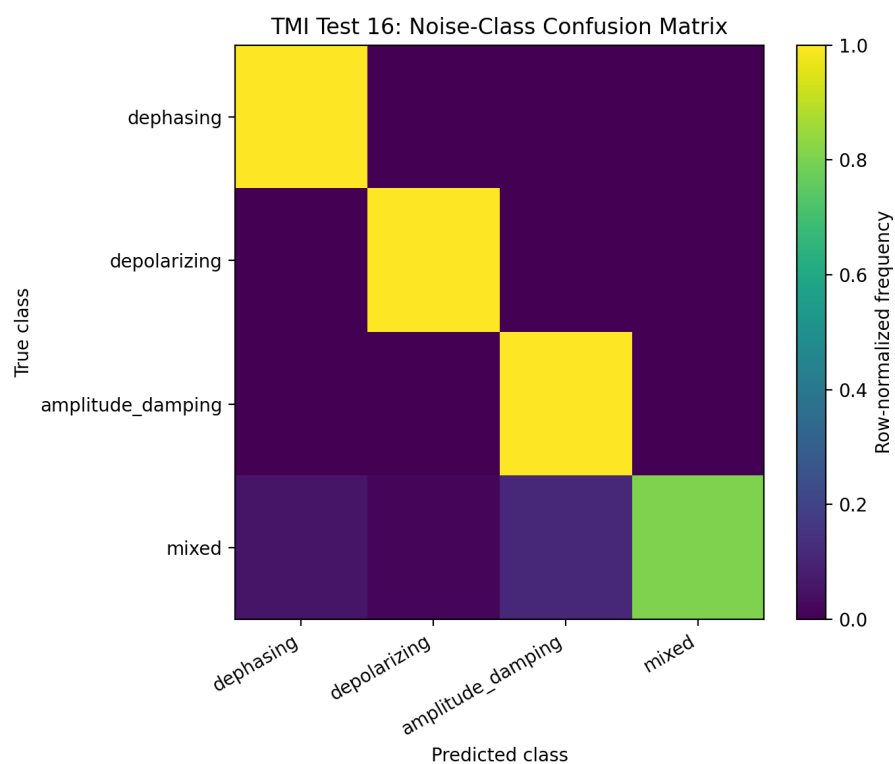


Рис. 5: Test 16: confusion matrix классификатора шума.

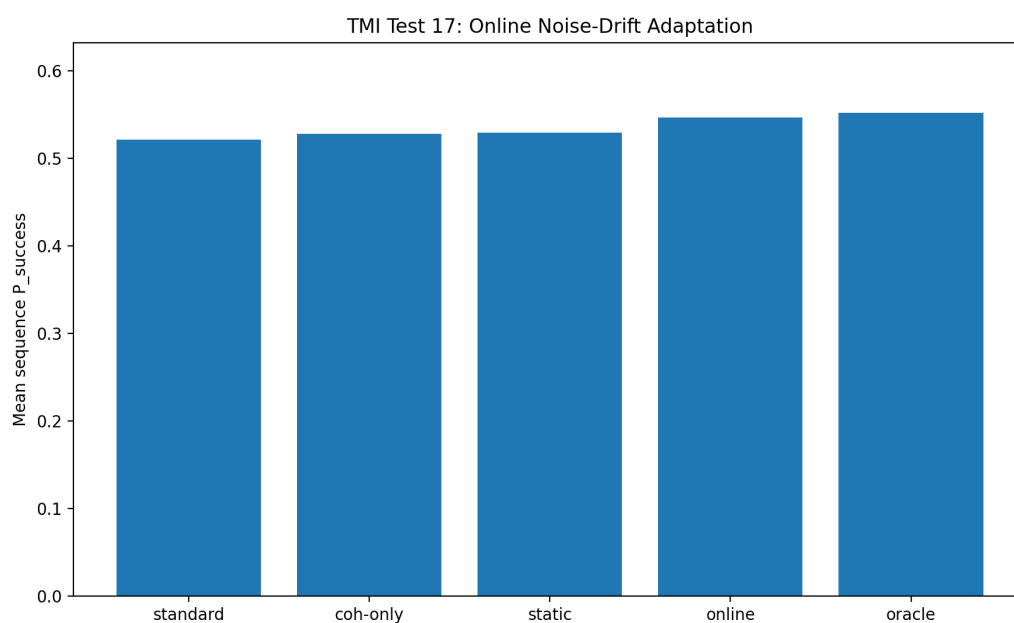


Рис. 6: Test 17: online classifier выигрывает у static classifier при noise drift.

6. Tests 18-19: задержка обратной связи и latency-aware управление

Test 18 проверяет устаревшее состояние:

$$NoiseClass(t - \Delta t) \Rightarrow ControlLayer(t).$$

Результат: задержка снижает качество online control:

$$P_{online} = 0.883, \quad P_{delayed\ d2} = 0.861.$$

Test 19 улучшает latency-aware слой через predictive distribution:

$$NoiseClass(t - \Delta t) \Rightarrow PredictiveNoiseDistribution(t) \Rightarrow ControlLayer(t).$$

После оптимизации:

$$P_{latency-opt} = 0.875 > P_{delayed\ d2} = 0.861.$$

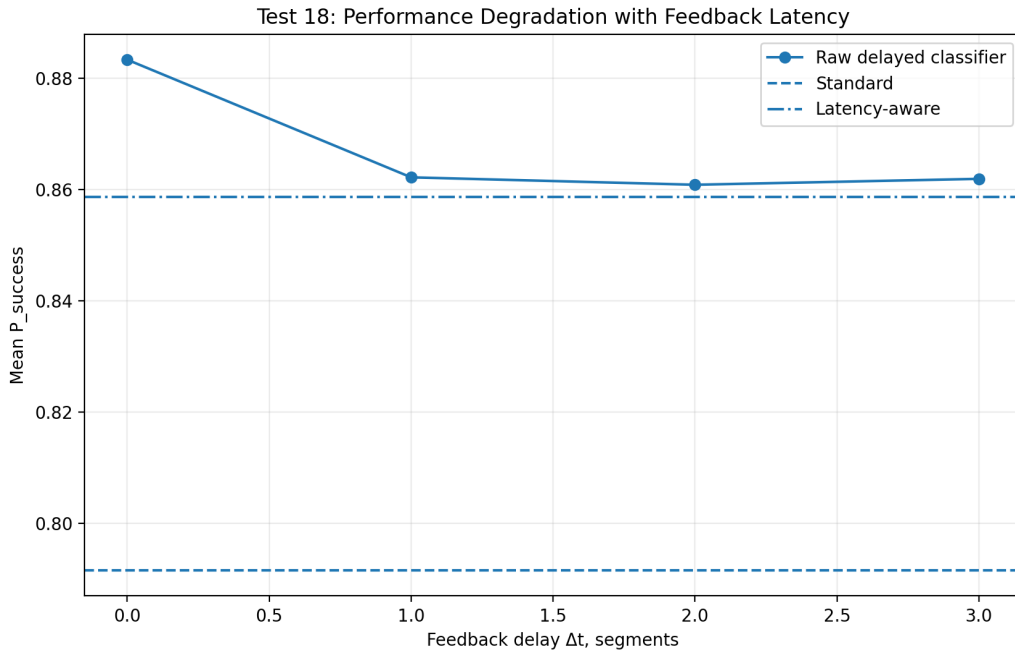


Рис. 7: Test 18: деградация качества при задержке feedback.

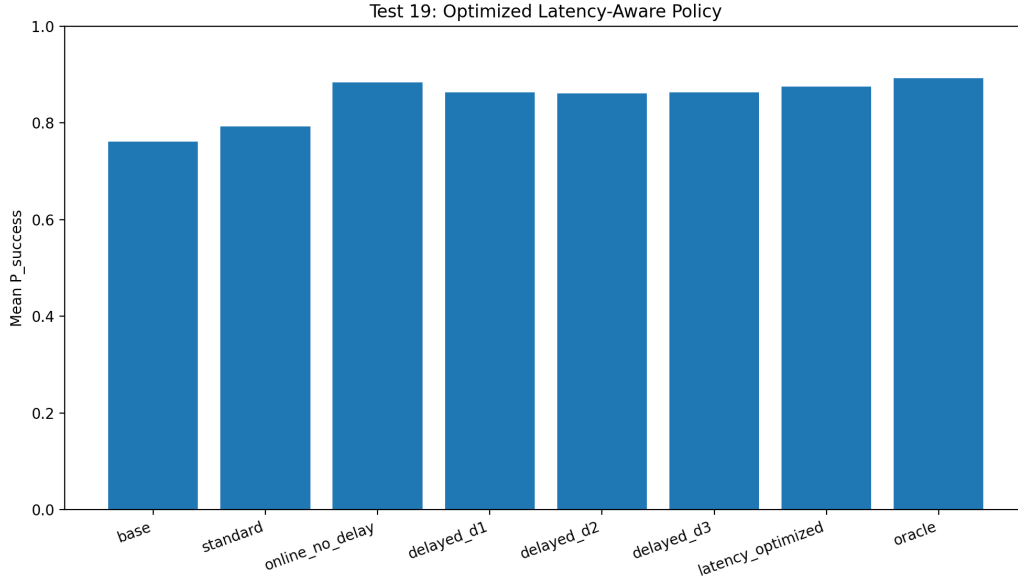


Рис. 8: Test 19: оптимизированный latency-aware слой превосходит raw delayed режимы.

7. Tests 20-22: от single metric к boundary manifold

7.1. Test 20: cost-overhead tradeoff

В Test 20 вводится цена управления:

$$Efficiency = \frac{P_{mode} - P_{standard}}{Cost_{mode} - Cost_{standard}},$$

$$Utility = \Delta P - 0.020 \cdot ExtraCost.$$

Результат: максимальная производительность не равна максимальной эффективности.

Policy	Mean P	Cost	ΔP	Efficiency	Utility
base	0.760	0.100	-0.0315	-	-0.0315
standard	0.791	0.700	0.0000	-	0.0000
cost_aware_low	0.865	1.680	0.0732	0.0747	0.0536
cost_aware_balanced	0.872	2.060	0.0808	0.0594	0.0536
latency_optimized	0.875	2.060	0.0833	0.0613	0.0561
high_perf_online	0.889	2.073	0.0979	0.0713	0.0705
oracle_high_cost	0.898	2.708	0.1068	0.0532	0.0666

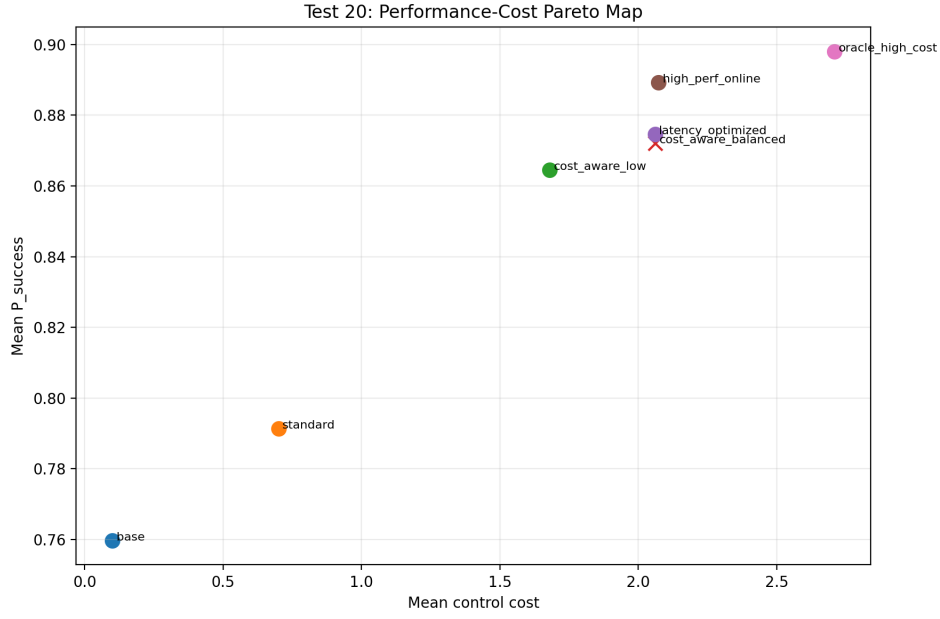


Рис. 9: Test 20: performance-cost Pareto map.

7.2. Test 21: cost-interface boundary

Test 21 находит критическую границу:

$$U_{hp} = U_{ca},$$

$$\lambda^* = \frac{\Delta P_{hp} - \Delta P_{ca}}{ExtraCost_{hp} - ExtraCost_{ca}}.$$

Главная boundary:

$$\lambda^* \approx 0.0636.$$

Если $\lambda < \lambda^*$, выгоднее high-performance. Если $\lambda > \lambda^*$, выгоднее cost-aware.

Boundary		Mean λ^*	Median λ^*	95% range
high_perf_online cost_aware_balanced	=	0.4652	0.3340	[0.0668, 2.3900]
high_perf_online cost_aware_low	=	0.0636	0.0640	[0.0240, 0.1121]
high_perf_online latency_optimized	=	0.3720	0.2900	[0.0306, 1.9437]
latency_optimized cost_aware_low	=	0.0265	0.0267	[0.0103, 0.0460]
oracle_high_cost cost_aware_low	=	0.0326	0.0328	[0.0182, 0.0485]
oracle_high_cost high_perf_online	=	0.0153	0.0133	[0.0011, 0.0384]

Это можно записать как интерфейс переключения:

$$B_{cost} : U_{hp} = U_{ca},$$

$$I_{cost} := B_{cost} \circ T(\text{Control}_{hp} \leftrightarrow \text{Control}_{ca}).$$

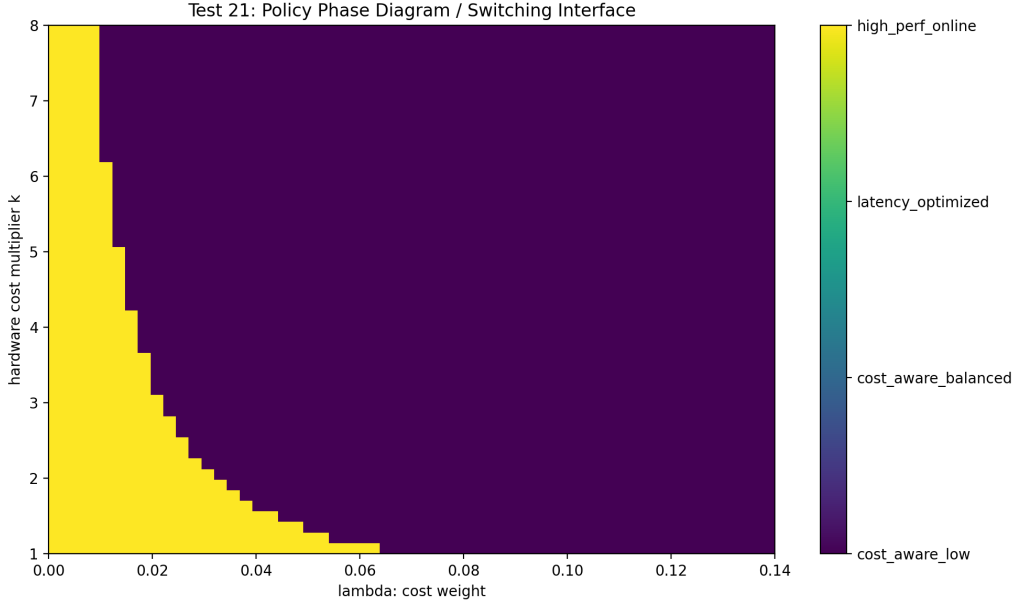


Рис. 10: Test 21: policy phase diagram; boundary становится интерфейсом переключения режимов.

7.3. Test 22: multi-objective boundary manifold

Test 22 обобщает одну границу на многообразие границ. Для политики:

$$O(policy) = (P, Cost, Safety, Latency, Robustness).$$

Для вектора предпочтений:

$$w = (w_P, w_C, w_S, w_L, w_R),$$

$$Score(policy; w) = w \cdot O(policy).$$

Граница между двумя режимами:

$$Score_i(w) = Score_j(w).$$

Совокупность границ:

$$\mathcal{M}_{boundary} = \{w : \arg \max_{policy} Score(policy; w) \text{ изменяется}\}.$$

Когда система использует это многообразие для переключения режимов, получаем:

$$I_{multi} := \mathcal{M}_{boundary} \circ T(\text{Control}_i \leftrightarrow \text{Control}_j).$$

Policy	P	Cost	Safety	Latency	Robustness
cost_aware_balanced	0.677	0.352	0.830	0.353	0.811
cost_aware_low	0.646	0.533	0.887	0.589	0.841
high_perf_online	0.749	0.346	0.632	0.255	0.692
latency_optimized	0.688	0.352	0.764	0.464	0.845
oracle_high_cost	0.786	0.044	0.593	0.031	0.836
standard	0.339	1.000	0.774	0.876	0.712

Winner share в sampled preference space:

$$standard = 0.608, \quad cost_aware_low = 0.373.$$

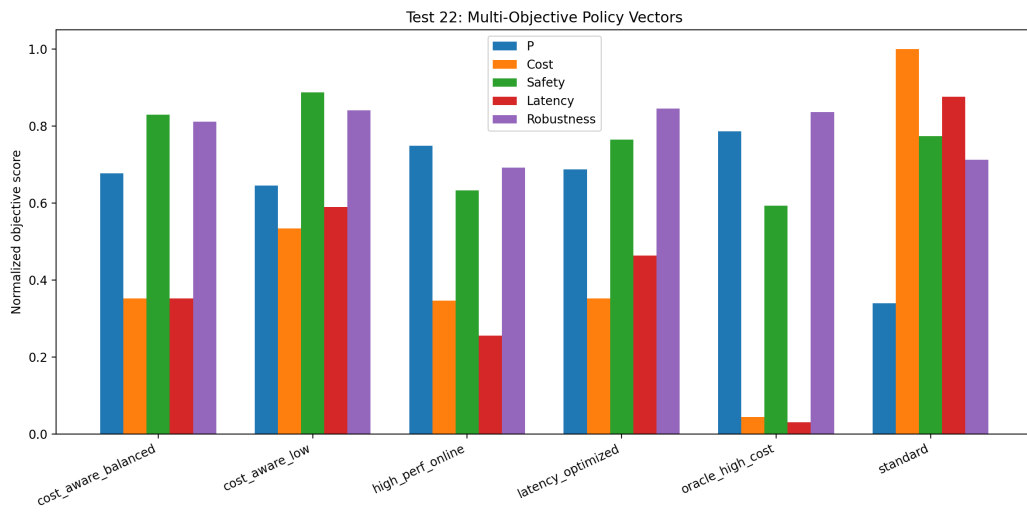


Рис. 11: Test 22: objective vectors по политикам управления.

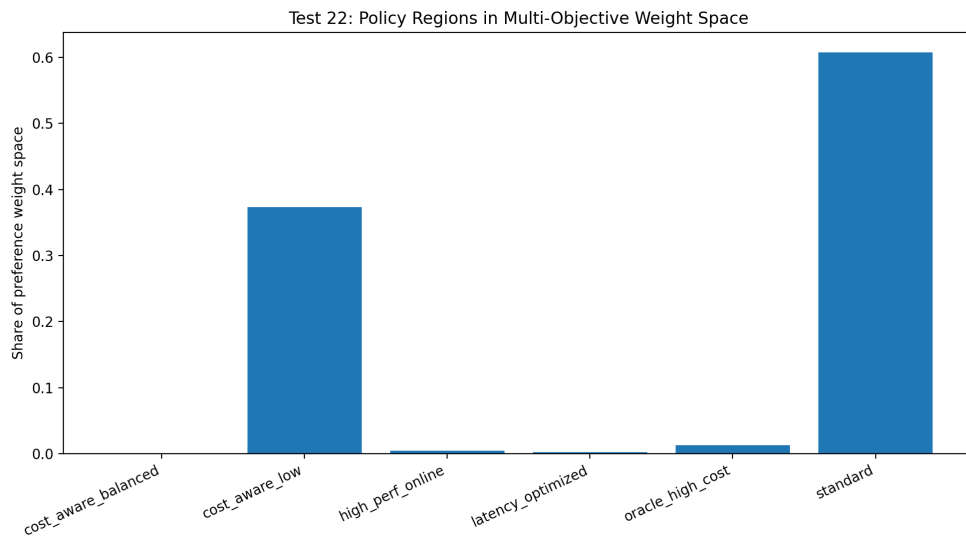


Рис. 12: Test 22: доля пространства предпочтений, где политика является лучшей.

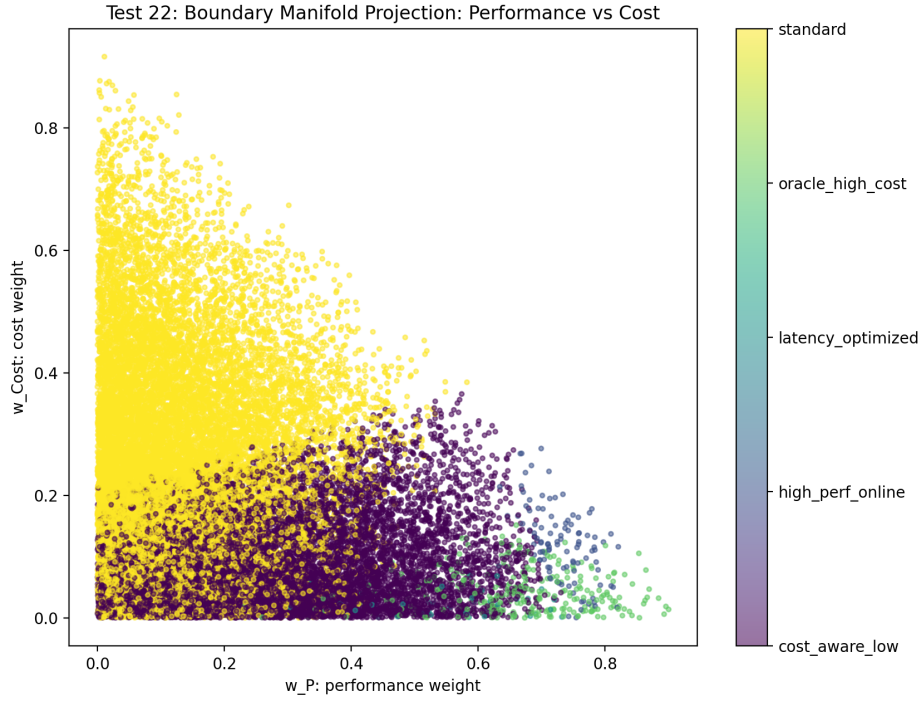


Рис. 13: Test 22: проекция $\mathcal{M}_{boundary}$ на веса performance/cost.

8. Интерпретация для Теории многообразных интерфейсов

Tests 1–19 строят интерфейс как адаптивную систему управления состоянием:

$$Input \Rightarrow Interpretation \Rightarrow ControlLayer \Rightarrow Output.$$

Tests 20–22 добавляют пространство целей и границ. Теперь интерфейс не просто выбирает параметр, а выбирает режим из множества допустимых режимов:

$$\{Control_i\}_{i \in \Lambda}.$$

Смена режима происходит на границах:

$$B_{ij} : Score_i(w) = Score_j(w).$$

Их совокупность образует многообразие границ:

$$\mathcal{M}_{boundary} = \bigcup_{i,j} B_{ij}.$$

Полная формула:

$$\mathcal{M}_{boundary} \circ T(Control_i \leftrightarrow Control_j) \Rightarrow I_{multi}$$

9. Главный вывод

Вычислительная линия теперь имеет три уровня:

Level 1: adaptive control Tests 1 – –13,

Level 2: noise-specific and drift-adaptive control *Tests* 14 – –19,

Level 3: boundary-manifold control *Tests* 20 – –22.

Сжатая итоговая формула:

$AutoInterface + NoiseInterpretation + CostBoundary + MultiObjectiveBoundaryManifold \Rightarrow Man$

10. Ограничения

1. Это вычислительная валидация, а не лабораторное подтверждение.
2. *Tests* 20–22 используют синтетическую модель cost/safety/latency/robustness.
3. Нужна независимая реализация полной линии на Qiskit/Cirq/QuTiP или аналогичном backend.
4. Нужна hardware-specific калибровка стоимости управления.
5. Multi-objective boundary manifold пока является вычислительной моделью, а не измеренным физическим объектом.

11. Следующие шаги

1. Зафиксировать *Tests* 1–22 как **Computational Validation Module 0.2**.
2. Подготовить английскую публикационную версию этого отчёта.
3. Сделать репозиторий с единым `run_all_tests.py` для расширенной линии 1–22.
4. Заменить synthetic cost/safety/latency на hardware-specific estimates.
5. После этого проектировать cloud-QPU protocol.

A. Состав архива

В архив входят:

- PDF и LaTeX исходник отчёта;
- figures/ – ключевые графики *Tests* 1–22;
- data/ – CSV/JSON/README summary-файлы;
- test_archives/ – доступные архивы отдельных тестов в текущем рабочем наборе; ранняя линия *Tests* 1–11 отражена в сводной таблице и предыдущем отчёте v2.